ColdCard hack: statistical analysis of the stolen funds

Course: Blockchain Analysis | **Assignment:** ColdCard – Statistical Analyses | **Work:** individual

1. Objective

The assignment gives a set of Bitcoin blocks (`blocos.tgz`) and a *collection address* to which stolen funds were sent:

`bc1qnk4zh9qcnap2mycp56qjrgza3cc8ylrh8fecp0`

The task is to identify the transactions that send BTC to the collection address, to assume that the addresses paying into it are being robbed, and to report at least: the amount stolen, from how many UTXOs, from how many addresses, the Gini coefficient of the stolen population compared with the blockchain, and the Benford (first-digit) distribution of the stolen population compared with the blockchain.

2. Data and method

Data. `blocos.tgz` (207 MB compressed, 1.64 GB uncompressed) holds 98 JSON files, one per block, in the Blockchain.com raw-block format. The blocks have heights between 960,179 and 960,802. They are **not contiguous** (98 blocks over a range of 624 heights), and the selection criterion is not stated in the assignment. Amounts in the files are in satoshis (1 BTC = 100,000,000 satoshis).

Parsing. Each block was read one at a time and flattened into three tables with pandas:

- `txs`: one row per transaction (hash, height, time, fee, size, weight, number of inputs and outputs). 493,571 rows.
- `ins`: one row per input, holding the address and value of the output being spent, and the identifier of that output (`prev_tx_index`, `prev_n`). 778,830 rows before cleaning.
- `outs`: one row per output (address, value, type, spent flag). 1,085,654 rows.

Validation. For every non-coinbase transaction, the sum of the inputs minus the sum of the outputs must equal the fee. The first check gave exactly 98 mismatches, one per block. These were the 98 coinbase transactions: their single input row has a value of 0 (apparently a placeholder, since a coinbase spends no earlier output), so the identity does not apply to them. After flagging them and removing their placeholder input rows (778,732 inputs remain), the check was repeated on the 493,473 remaining transactions and gave **0 mismatches**.

Identifying the theft. A transaction is a candidate theft if it has an output to the collection address. The inputs of such a transaction are taken to be the stolen coins, and the addresses behind those inputs the victims. Transactions that also spend from the collection address would be self-transfers and are excluded. There are none among the payments (see below).

3. Results: the theft

Quantity	Value
Transactions paying the collection address	504 (504 outputs, one per transaction)
Block heights of these transactions	960,188 to 960,793
Value received by the collection address (stolen amount)	**594.47725695 BTC**
Value of the stolen inputs	594.51489212 BTC
Fees paid in these transactions	0.03659231 BTC
Other outputs (change or other payments)	0.00104286 BTC
Stolen UTXOs	**1,329** (all distinct)
Victim addresses	**504** (no missing addresses)
Transactions with a single output (full sweep)	501 of 504 (the other three have 2 or 3 outputs)
Inputs per transaction	median 1, mean 2.64, maximum 200

The stolen amount is reported as the value received by the collection address, since that is what the thief obtained. The victims' inputs total is slightly higher because of fees (0.0366 BTC) and the extra outputs in three transactions (0.00104 BTC): 594.51489212 − 0.03659231 − 0.00104286 = 594.47725695.

The collection address. It receives all 504 payments and spends in exactly one transaction, with 341 inputs. There are no self-transfers. *Hypothesis (not tested):* that transaction consolidates most of the stolen coins. Where it sends them was not analysed.

Amounts per victim address (value of the stolen inputs, per address):

Statistic	Value
Mean	1.180 BTC
25th percentile / median / 75th percentile	0.238 / 0.406 / 1.000 BTC
Maximum	29.89 BTC
Minimum	0.00001092 BTC (1,092 satoshis)

The five largest victims lost 29.89, 24.08, 14.43, 14.25 and 11.74 BTC, which is 94.39 BTC or 15.9% of the stolen input value.

4. Gini coefficient

The Gini coefficient measures inequality: 0 means all values are equal and values near 1 mean that a few hold almost everything. For sorted values $x_1 \le \dots \le x_n$,

$G = 2 \cdot \Sigma(i \cdot x_i) / (n \cdot \Sigma x_i) - (n+1)/n$.

The implementation was tested on `[1,1,1,1]` (0.0) and `[0,0,0,1]` (0.75), as expected. Since balances are not available, the quantity compared is the **total value each address spent** in the provided blocks. For the stolen population this is the amount taken from each victim.

Population	Addresses	Gini
Stolen (victims)	504	**0.666**
All addresses in the provided blocks	304,569	0.9946
All addresses except the victims	304,065	0.9947

Because the two groups differ greatly in size, the comparison was repeated with 1,000 random samples of 504 addresses drawn from the non-victim population. Their Gini coefficients have a mean of 0.962 and a 95% range of 0.920 to 0.997. The stolen value of 0.666 lies far outside this range.

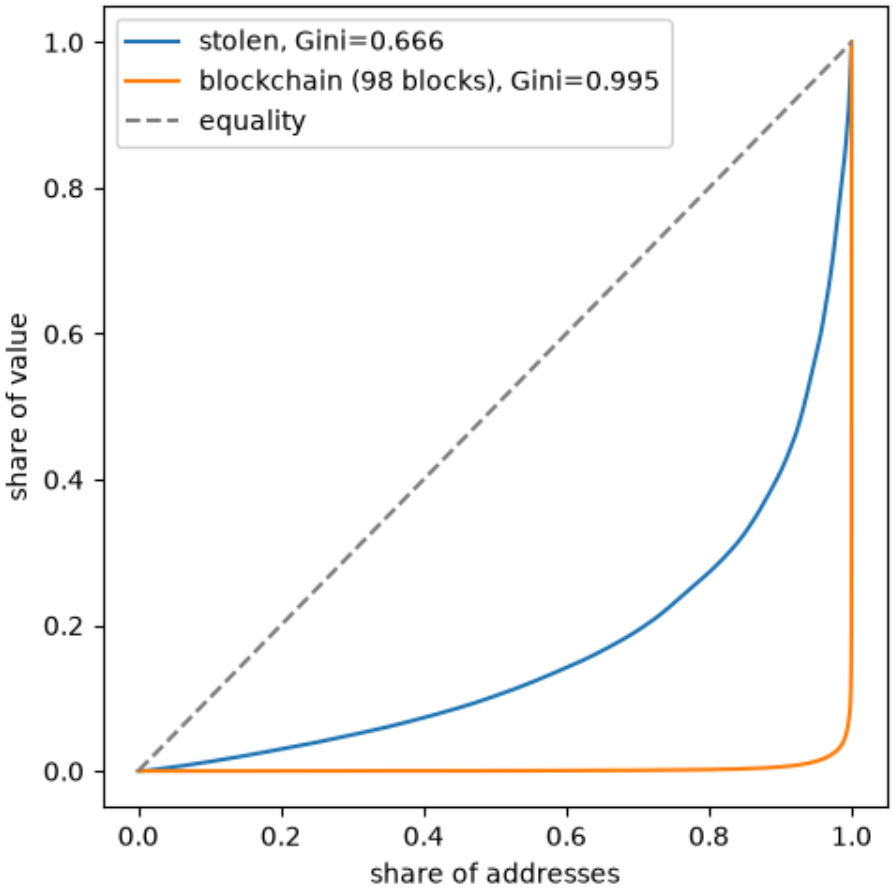


Figure 1. Lorenz curves of the stolen population and of all addresses in the provided blocks.

Reading. The stolen population is much less unequal than the blockchain as a whole, whose per-address amounts are extremely skewed (a few very large movers and a long tail of tiny amounts). Removing the 504 victims leaves the blockchain figure essentially unchanged. *Hypothesis (not tested):* the victims resemble ordinary holders and not services or very large holders.

5. Benford's law

Benford's law gives the expected share of first digit d as $\log_{10}(1 + 1/d)$: 30.1% for digit 1, 17.6% for 2, down to 4.6% for 9. It tends to hold for amounts spanning several orders of magnitude. The first digit of each **UTXO value** was compared, for the 1,329 stolen UTXOs and for the 777,403 other inputs in the blocks. Two measures are used: the mean absolute deviation (MAD) between observed and expected shares (Nigrini's scale for first digits: below 0.006 close conformity, 0.006–0.012 acceptable, 0.012–0.015 marginal, above 0.015 non-conformity) and a chi-square test. With hundreds of thousands of values, the chi-square test rejects for any small deviation, so MAD is the more informative measure for the large population.

Digit	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Benford expected	.301	.176	.125	.097	.079	.067	.058	.051	.046
Stolen UTXOs	.419	.223	.076	.047	.105	.068	.022	.017	.022
Other inputs	.303	.152	.141	.096	.103	.070	.055	.039	.040

Population	n	MAD	Verdict	Chi-square
Stolen UTXOs	1,329	**0.0428**	non-conformity	225.0 (8 d.f.), p ≈ 3.4×10^{-44}
Other inputs	777,403	**0.0101**	acceptable conformity	12,868 (p ≈ 0, not informative at this n)

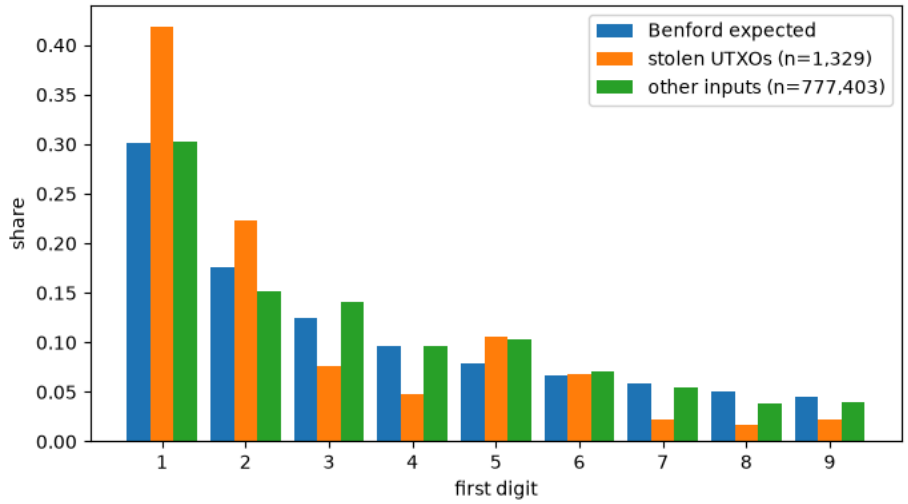


Figure 2. First-digit shares: Benford expectation, stolen UTXOs and other inputs.

Reading. The rest of the blockchain is close to Benford, with small deviations (a surplus of digit 5 and a deficit of digit 2). The stolen UTXOs depart clearly from it: 64.2% start with 1 or 2, against 47.7% expected, and too few start with 7, 8 or 9.

Check of a possible explanation. The spread of the two populations is the same (standard deviation of $\log_{10}$(value in satoshis) of 1.27 in both), so a narrower range does not explain the difference. What differs is the location and the lower tail:

$\log_{10}$(value in satoshis)	5th pct.	25th	median	75th	95th
Stolen UTXOs	5.00	5.78	6.36	7.42	8.26
Other inputs	2.58	4.20	4.80	5.61	7.18

Stolen UTXOs are about 36 times larger at the median (about 0.023 BTC against 0.0006 BTC) and have almost no small values: their 5th percentile is about 0.001 BTC, while 5% of other inputs are below roughly 380 satoshis. *Hypothesis (not tested):* a hard lower floor near 100,000 satoshis can by itself produce an excess of leading 1s and 2s. The deviation is reported as a measured departure from Benford; it is not evidence of manipulated amounts.

6. Limitations

- **Block selection.** The 98 provided blocks are not contiguous (heights 960,179 to 960,802) and their selection criterion is unknown. If they were chosen because they contain theft-related transactions, the "blockchain" baseline for the Gini and Benford comparisons is not a random sample of the blockchain. This was not verified.
- **Short window, no balances.** The Gini coefficient compares amounts *spent per address* inside these blocks, not balances or wealth.
- **Assumption on ownership.** As stated in the assignment, every address that pays the collection address is treated as a victim. Nothing in the data confirms that each payment was a theft.
- **Dependence.** The 1,329 stolen UTXOs come from 504 addresses, so they are not independent observations, which weakens the chi-square test for the stolen group.

- **Sample size.** The two populations differ greatly in size (504 against about 304,000 addresses, 1,329 against about 777,000 UTXOs); the 504-address resampling addresses this for the Gini coefficient but not for Benford.
- **Untested explanations.** The statements marked as hypotheses (consolidation by the thief, ordinary holders, a lower floor near 100,000 satoshis) were not tested.
- **Not analysed:** where the collection address sends the funds, and the three transactions with extra outputs.

Appendix A. Code

The code ran as a sequence of notebook cells in Python (pandas, numpy, scipy, matplotlib). Cells are given in the order they were run.

A.1 Parsing the blocks into tables

```
import tarfile, json, os
import pandas as pd, numpy as np

COLLECT = "bc1qnk4zh9qcnap2mycp56qjrgza3cc8ylrh8fecp0"

tx_rows, in_rows, out_rows = [], [], []
with tarfile.open("blocos.tgz", "r:gz") as t:
    for m in t:
        if not m.isfile():
            continue
        blk = json.load(t.extractfile(m))
        h = blk["height"]
        for tx in blk["tx"]:
            th = tx["hash"]
            tx_rows.append((th, h, tx.get("time"), tx.get("fee"), tx["size"],
                            tx["weight"], tx["vin_sz"], tx["vout_sz"],
                            tx["lock_time"], tx["ver"]))
            for i, vin in enumerate(tx["inputs"]):
                po = vin.get("prev_out")
                if po is None:
                    continue
                in_rows.append((th, i, vin["sequence"], po.get("addr"),
                                po["value"], po.get("type"), po["tx_index"], po["n"]))
            for o in tx["out"]:
                out_rows.append((th, o["n"], o.get("addr"), o["value"],
                                 o.get("type"), o.get("spent")))

txs = pd.DataFrame(tx_rows, columns=["txid", "height", "time", "fee", "size",
                                     "weight", "n_in", "n_out", "lock_time", "ver"])
ins = pd.DataFrame(in_rows, columns=["txid", "idx", "sequence", "addr", "value",
                                     "type", "prev_tx_index", "prev_n"])
outs = pd.DataFrame(out_rows, columns=["txid", "n", "addr", "value", "type", "spent"])

os.makedirs("data", exist_ok=True)
txs.to_pickle("data/txs.pkl"); ins.to_pickle("data/ins.pkl"); outs.to_pickle("data/outs.pkl")
print(txs.shape, ins.shape, outs.shape)          # (493571, 10) (778830, 8) (1085654, 6)
```

A.2 Validation: inputs − outputs = fee, and coinbase clean-up

```
sum_in  = ins.groupby("txid")["value"].sum()
sum_out = outs.groupby("txid")["value"].sum()
chk = txs.set_index("txid").join(sum_in.rename("sum_in")).join(sum_out.rename("sum_out"))
chk = chk[chk["sum_in"].notna()]
bad = chk[(chk["sum_in"] - chk["sum_out"]) != chk["fee"]]
print(len(chk), len(bad))                        # 493571 98
first_tx = set(txs.groupby("height")["txid"].first())
print(bad.index.isin(first_tx).all(), (bad["sum_in"] == 0).all())   # True True

cb = set(bad.index)                              # the 98 coinbase transactions
txs["is_coinbase"] = txs["txid"].isin(cb)
ins = ins[~ins["txid"].isin(cb)].copy()
```

```
sum_in  = ins.groupby("txid")["value"].sum()
sum_out = outs.groupby("txid")["value"].sum()
chk =
txs[~txs["is_coinbase"]].set_index("txid").join(sum_in.rename("sum_in")).join(sum_out.rename("sum_out"))
print(len(chk), ((chk["sum_in"] - chk["sum_out"]) != chk["fee"]).sum())   # 493473 0
```

A.3 Identifying the theft transactions

```
paid = outs[outs["addr"] == COLLECT]
print(len(paid), paid["txid"].nunique(), paid["value"].sum() / 1e8)       # 504 504 594.47725695

collect_spends = set(ins.loc[ins["addr"] == COLLECT, "txid"])
paid_tx = set(paid["txid"])
self_tx   = paid_tx & collect_spends
inflow_tx = paid_tx - collect_spends
print(len(collect_spends), len(self_tx), len(inflow_tx))                   # 1 0 504

pt = txs[txs["txid"].isin(inflow_tx)]
print(pt["height"].min(), pt["height"].max())                              # 960188 960793
print(pt["n_in"].describe())
print(pt["n_out"].value_counts())                                          # 1: 501, 3: 2, 2: 1
```

A.4 Victims: UTXOs, addresses, values

```
victim_ins = ins[ins["txid"].isin(inflow_tx)].copy()

print(len(victim_ins))                                                     # 1329
print(victim_ins[["prev_tx_index", "prev_n"]].drop_duplicates().shape[0])  # 1329
print(victim_ins["addr"].nunique(), victim_ins["addr"].isna().sum())       # 504 0
print(victim_ins["value"].sum() / 1e8)                                     # 594.51489212
print(txs[txs["txid"].isin(inflow_tx)]["fee"].sum() / 1e8)                 # 0.03659231

stolen_by_addr = victim_ins.groupby("addr")["value"].sum().sort_values(ascending=False)
print(stolen_by_addr.describe())
print((stolen_by_addr / 1e8).head(5))
```

A.5 Gini coefficient

```
def gini(x):
    x = np.sort(np.asarray(x, dtype=float))
    n = len(x)
    return (2 * np.sum(np.arange(1, n + 1) * x)) / (n * x.sum()) - (n + 1) / n

print(gini([1, 1, 1, 1]), gini([0, 0, 0, 1]))                              # 0.0 0.75

spent_all    = ins.groupby("addr")["value"].sum()
spent_others = ins[~ins["txid"].isin(inflow_tx)].groupby("addr")["value"].sum()
print(len(stolen_by_addr), gini(stolen_by_addr.values))                    # 504 0.666
print(len(spent_all),      gini(spent_all.values))                         # 304569 0.9946
print(len(spent_others),   gini(spent_others.values))                      # 304065 0.9947

rng = np.random.default_rng(0)
vals = spent_others.values
gs = [gini(rng.choice(vals, 504, replace=False)) for _ in range(1000)]
print(np.mean(gs), np.percentile(gs, [2.5, 97.5]))                         # 0.962 [0.920 0.997]
```

A.6 Benford's law

```
from scipy import stats

exp = np.log10(1 + 1 / np.arange(1, 10))

def first_digit(v):
    v = np.asarray(v, dtype=np.int64)
    v = v[v > 0]
    return np.array([int(str(x)[0]) for x in v])

def benford(values):
    d = first_digit(values)
    counts = np.bincount(d, minlength=10)[1:]
```

```
    obs = counts / len(d)
    mad = np.abs(obs - exp).mean()
    chi, p = stats.chisquare(counts, f_exp=exp * len(d))
    return obs, mad, chi, p, len(d)

stolen_vals = victim_ins["value"]
other_vals  = ins.loc[~ins["txid"].isin(inflow_tx), "value"]

print("expected", np.round(exp, 3))
for name, v in [("stolen", stolen_vals), ("others", other_vals)]:
    obs, mad, chi, p, n = benford(v)
    print(name, n, "MAD", round(mad, 4), "chi2", round(chi, 1), "p", p)
    print(np.round(obs, 3))

lv = np.log10(stolen_vals[stolen_vals > 0])
lo = np.log10(other_vals[other_vals > 0])
print(np.percentile(lv, [5, 25, 50, 75, 95]).round(2), lv.std().round(2))
print(np.percentile(lo, [5, 25, 50, 75, 95]).round(2), lo.std().round(2))
```

A.7 Figures

```
import matplotlib.pyplot as plt
os.makedirs("figures", exist_ok=True)

digits = np.arange(1, 10)
o_s = benford(stolen_vals)[0]
o_o = benford(other_vals)[0]
w = 0.28
fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(digits - w, exp, w, label="Benford expected")
ax.bar(digits,     o_s, w, label="stolen UTXOs (n=1,329)")
ax.bar(digits + w, o_o, w, label="other inputs (n=777,403)")
ax.set_xlabel("first digit"); ax.set_ylabel("share"); ax.set_xticks(digits); ax.legend()
fig.tight_layout(); fig.savefig("figures/benford.png", dpi=200)

def lorenz(x):
    x = np.sort(np.asarray(x, float))
    c = np.cumsum(x) / x.sum()
    return np.linspace(0, 1, len(x) + 1), np.insert(c, 0, 0)

fig, ax = plt.subplots(figsize=(5, 5))
for name, v in [("stolen", stolen_by_addr.values), ("blockchain (98 blocks)", spent_all.values)]:
    p, c = lorenz(v)
    ax.plot(p, c, label=f"{name}, Gini={gini(v):.3f}")
ax.plot([0, 1], [0, 1], "--", color="gray", label="equality")
ax.set_xlabel("share of addresses"); ax.set_ylabel("share of value"); ax.legend()
fig.tight_layout(); fig.savefig("figures/lorenz.png", dpi=200)
```